Short-Term Hourly Rainfall Prediction: A Hybrid Approach with LSTM and TimeGAN

abstract

Short-term rainfall prediction is crucial in regions like West Africa, where unpredictable climate fluctuations between droughts and heavy rainfall complicate water and resource management. Accurate hourly forecasts can enhance water allocation for agriculture, drinking water, and irrigation, while improving disaster preparedness for floods and droughts. This study introduces a hybrid approach using LSTM networks and TimeGAN to address traditional forecasting limitations. By generating synthetic data with TimeGAN, the model tackles data scarcity and improves prediction robustness. Results show that models trained on synthetic data perform as well as or better than those trained on real data, demonstrating the value of synthetic data augmentation. This approach has significant potential for water management, agriculture, and disaster mitigation, offering scalable solutions to build resilience against extreme weather. The study highlights the effectiveness of combining generative models with recurrent neural networks for accurate, adaptive forecasting systems tailored to local needs.

1. Introduction

Short-term rainfall prediction is crucial in regions with variable climates like West Africa, where sudden climatic fluctuations make water management complex [1]. Accurate forecasts improve water allocation for agriculture, drinking supply, and irrigation while aiding disaster management by allowing timely preventive measures [2, 3]. This is especially important in developing countries with limited infrastructure and risk management systems [4].

Reliable forecasts contribute to food security by helping farmers make informed decisions on planting, harvesting, and irrigation [5]. Predicting hourly rainfall enables better planning, reducing agricultural losses [6]. Additionally, infrastructure management, including reservoirs and dams, benefits from precise forecasts to prevent overflow or shortages [7].

This paper explores the importance of hourly rainfall prediction using LSTM neural networks and generative models like TimGAN. Combining these models generates additional data, improving prediction accuracy for high-frequency climatic events [8]. The research examines how synthetic data enhances forecast reliability and reduces uncertainty in unstable climates [9].

2. State of the Art

Traditional rainfall prediction models like ARIMA have been used for short-term forecasting but struggle with nonlinear and complex meteorological data [10]. Machine learning models, particularly LSTM and GRU, offer improved predictive capabilities by capturing complex temporal relationships [11]. However, they face challenges such as overfitting on small datasets and difficulty in generalizing to extreme weather events [12].

To address these issues, integrating generative models like TimGAN can enhance performance by generating synthetic data, increasing dataset diversity, and improving robustness [13]. A hybrid approach combining TimGAN with LSTM or GRU has been proposed to strengthen predictive capacity while minimizing overfitting risks [14].

Hourly rainfall prediction presents additional challenges due to rapid fluctuations and data uncertainty. Hybrid models can enhance high-frequency forecasts by leveraging recurrent networks' ability to capture temporal trends and generative models' flexibility in handling uncertainties [13]. In regions like West Africa, where data collection is limited, these models can improve accuracy, benefiting water management, disaster prevention, and agriculture. Their integration could significantly impact decision-making, ensuring better preparedness for extreme weather events and reducing climate-related risks [11].

3. Data

Our study is conducted in the West African region, which consists of three geographical sub-domains corresponding to the three major West African climatic zones: the Guinean zone (approximately 6°-8°N), the Sudanian zone (approximately 8°-12°N), and the Sahelian zone (approximately 12°-16°N). These weather parameters are divided into three categories: Geographical coordinates: longitude [-20W; 10E] and latitude [17N; 5S]; date (time) in the format year-month-day and hour; pressure levels (level) [100hpa, 200hpa, 600hpa, 700hpa, 825hpa, 1000hpa]. These pressure levels correspond to the levels of precipitation formation. Thus, the variables used in this study are relative humidity, specific humidity, the eastward wind component, the northward wind component, all measured at different

levels of precipitation, total precipitation, sea surface temperature, temperature at 2 meters above the ground, and pressure level.

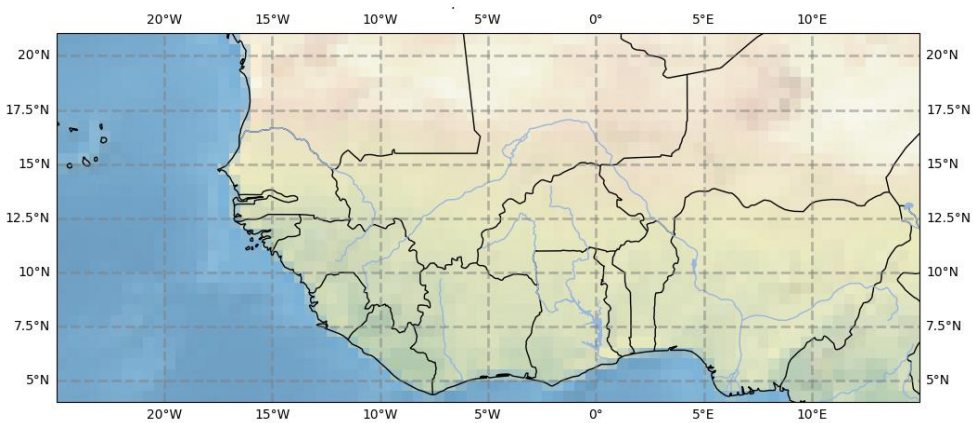

Figure 1: Map of West Africa

4.Methodology

The methodology of this project is based on the use of deep learning algorithms, particularly recurrent neural networks (RNN) and their advanced variants such as Long Short-Term Memory (LSTM) networks, Gated Recurrent Unit (GRU) networks, and Bidirectional LSTM (BiLSTM) networks.

Deep learning models were chosen because of their ability to handle sequential data and capture long-term dependencies. RNNs are particularly suited for sequence modeling as they can take previous information into account during their prediction process.

4.1 RNN, LSTM, BiLSTM, and GRU

4.1.1 Recurrent Neural Networks (RNN)

Before introducing LSTM and GRU networks, we first describe the structure of the RNN model, as LSTM and GRU networks are updated versions of RNNs with the same fundamental framework. An RNN consists of a series of modules with the same basic structure. Figure 2 shows the basic structure of a traditional RNN. ***xt*** is the input vector at time step ***t***. ***ht*** is the final output of the RNN cell at time step ***t***. The term ht in the RNN refers to a hidden state. A hidden state (***ht***) is calculated based on the hidden state (***ht−1***) at the previous time step ***t−1*** and the input vector (***xt***) at the current time step ***t***. The specific update equation for a hidden state (***ht***) is explained as follows:

$$h_t = tanh(U_{x_t} + W_{h_{t-1}} + b) \qquad (1)$$

where tanh is an activation function named hyperbolic tangent, b is a bias vector; the weight matrices ***U*** and ***W*** are parameters for the hidden state h and the input ***x***, respectively [15]. For each time step, the corresponding basic structures have the same parameters, namely ***b, U,*** and ***W***.

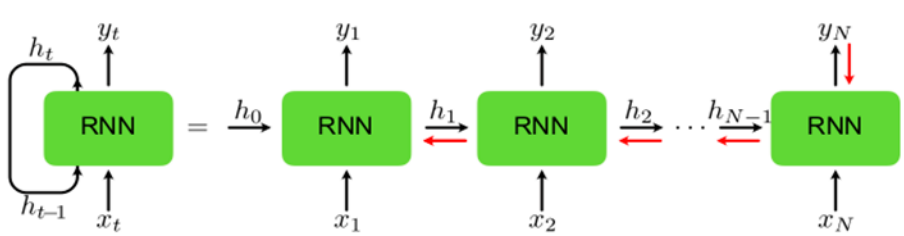

Figure 2*: The traditional RNN structure: Basic compressed RNN (left) and unfolded (right)*

The final output of the RNN depends not only on the input of the current time step but also on the information computed from the hidden layer during the previous time step [16]. In theory, the RNN can leverage all information, regardless of the sequence length. However, due to the vanishing and exploding gradient problems, the accuracy of RNN models decreases when the temporal sequence length becomes too long. LSTM and GRU networks were proposed to address these issues [17].

4.1.2 Long Short-Term Memory (LSTM)

Long Short Term Memory (LSTM) blocks were invented by by Hochreiter and Schmidhuber in 1997 [18] as an alternative to classical recurrent blocks. LSTM neural networks have a chain structure similar to traditional RNNs, but the internal operations within LSTM cells are more complex, enabling LSTMs to learn long-term dependencies. Unlike traditional RNNs, LSTM networks have an additional cell state where information can be stored. The basic structure of an LSTM cell is illustrated in Figure 5, where ***xt*** is the input vector, ***ht(ht−1)*** is the hidden state of the LSTM cell at time step ***t (t−1)***, and ***ct (ct−1)*** is the cell state of the LSTM cell at time step ***t(t−1)***. The structure of the LSTM cell shows that its cell state ***ct*** and hidden state ***ht*** are transferred to the next time step. The LSTM cell has three gates that maintain and adjust the cell state ***ct*** and hidden state ***ht***, namely a forget gate ***ft***, an input gate ***it***, and an output gate ***ot***. These three gates act as filters, serving different purposes [19]. The forget gate ***ft*** determines which information will be removed from the cell state ***ct***. The input gate ***it*** determines which new information will be stored in the cell state ***ct***. The output gate ***ot*** specifies which information from the cell state is used as output ***ot***. Through the functions of the three gates, LSTM cells can capture complex correlations in both short-term and long-term time series, which represents a significant improvement over traditional RNNs [16]. In Figure 5, the cell state ***ct*** and the hidden state ***ht*** of the LSTM cell are calculated as follows:

$$f_t = \sigma\,(W_{xf}\,x_t + W_{hf}h_{t-1} + b_f\,) \tag{2}$$

$$i_t = \sigma\,(W_{xi}\,x_t + W_{hi}h_{t-1} + b_i) \tag{3}$$

$$\tilde{C}_t = \tanh(W_{xc}\,x_t + W_{hc}h_{t-1} + b_c) \tag{4}$$

$$c_t = f_t \otimes c_{t-1} + i_t \otimes \hat{c}_t \tag{5}$$

$$o_t = \sigma\,(W_{xo}\,x_t + W_{ho}h_{t-1} + b_o) \tag{6}$$

$$h_t = o_t \tanh \otimes c_t \tag{7}$$

where σ is the logistic sigmoid function, ⊗ represents the element-wise multiplication of two vectors, and W_{xf}, W_{xi}, W_{hi}, W_{hf}, W_{xo}, W_{ho}, W_{xc}, and W_{hc} are the weight matrices of the network. Similarly, b_i, b_f, b_o, and b_c are bias vectors. Finally, f_t, i_t, and o_t are vectors corresponding to the activation values of the forget gate, input gate, and output gate, respectively.

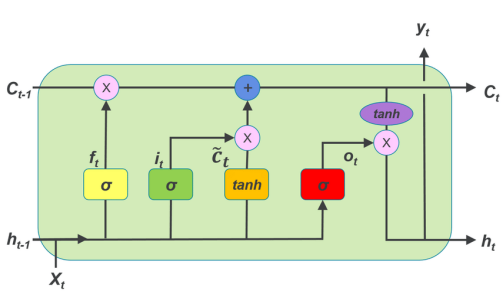

Figure 3 Description of an LSTM Cell

4.1.3 Bidirectional Long Short-Term Memory (BiLSTM)

In general time series processing, LSTM often ignores future information. BiLSTM uses two distinct hidden layers to process time series data in both forward and backward directions based on LSTM, connecting the two hidden layers

to the same output layer [20], storing both past and future information as the current temporal basis of the time series data [21].
Therefore, the theoretical prediction performance is better than that of unidirectional LSTM. The output of the hidden layer in BiLSTM includes the activation outputs of both the forward and backward hidden layers. The expression of the BiLSTM is presented in formulas (8), (9), and (10), where σ is the activation function, ***Ht*** is the input to the hidden layer, and the output is generated by updating the forward structure $\overrightarrow{\boldsymbol{h_t}}$ and the backward structure $\overleftarrow{\boldsymbol{h_t}}$. The structure of the BiLSTM is illustrated in Figure 4.

$$\overrightarrow{h_t} = \sigma\,(W_{x\overrightarrow{h_t}}\, x_t \;+\; W_{\overrightarrow{h_t}\overrightarrow{h_t}}\overrightarrow{h_{t-1}} \;+\; b_{\overrightarrow{h_t}}) \tag{8}$$

$$\overleftarrow{h_t} = \sigma\,(W_{x\overleftarrow{h_t}} x_t \;+\; W_{\overleftarrow{h_t}\overleftarrow{h_t}}\overleftarrow{h_{t-1}} \;+\; b_{\overleftarrow{h_t}}) \tag{9}$$

$$H_t = W_{x\vec{h}}\vec{h} \;+\; W_{\overleftarrow{h}y}\vec{h} + b_y \tag{10}$$

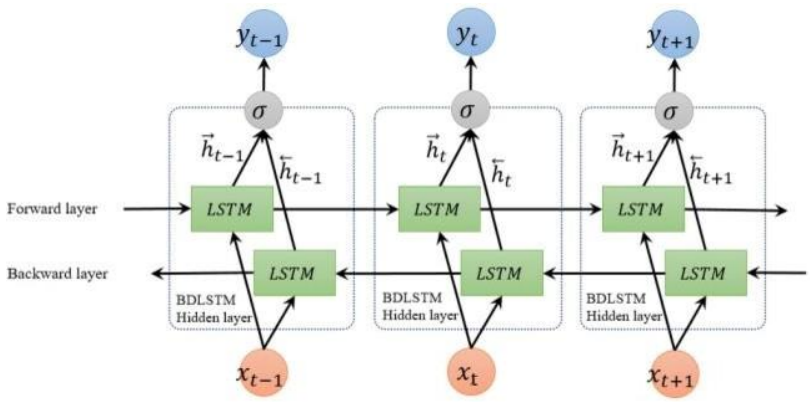

Figure 4: Description of a BiLSTM Cell

4.1.4 Gated Recurrent Unit (GRU)

A Gated Recurrent Unit (GRU) network is a variation of LSTM introduced in 2014 by Kyunghyun Cho et al. [22]. Indeed, LSTM neural networks have a complex structure and their training process generally takes a long time. To speed up the training process, GRU networks were proposed as a modification of LSTM networks with a simpler structure. The specific structure of the GRU cell is presented in Figure 5. The hidden state (**h_t**) and the cell state (**c_t**) are merged in the GRU. In the GRU cell, there are two control gates: the update gate (**z_t**) and the reset gate (**r_t**). The update gate (**z_t**) is used to control the extent to which information from the previous time step's state ***h_{t-1} (c_{t-1})*** will be incorporated into the current time step t. The higher the value of the update gate, the more information from the previous time step's state is integrated. The reset gate ***(r_t)*** controls the amount of information from the previous state in the current candidate set **c_t**. The smaller the reset gate (**r_t**), the less information from the previous state is written. In the GRU cell structure illustrated in Figure 4, the update equations are calculated as follows:

$$r_t = \sigma\,(W_{xr}\, x_t \;+\; W_{hr} h_{t-1} \;+\; b_r) \tag{11}$$

$$z_t = \sigma\,(W_{xz}\, x_t \;+\; W_{hz} h_{t-1} \;+\; b_z) \tag{12}$$

$$\hat{c}_t = \tanh(W_{xc}\, x_t \;+\; W_{hc}(\, r_t \otimes h_{t-1}) + \; b_c) \tag{13}$$

$$c_t = (1 - z_t) \otimes \; c_{t-1} \;+\; z_t \otimes \; \hat{c}_t \tag{14}$$

$$h_t = c_t \tag{15}$$

Where W_{xr}, W_{hr}, W_{xz}, and W_{hz} are the weight matrices of the network. b_r and b_z are the bias vectors, and r_t and z_t are the vectors of the activation values of the update gate and the reset gate.

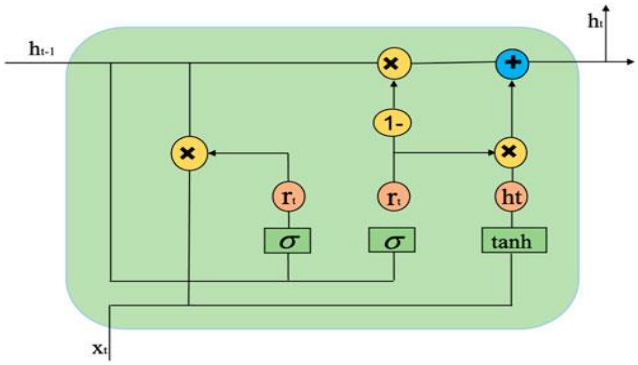

Figure 5: Description of a GRU Cell

4.2 Generative Adversarial Networks for Time Series (TimeGAN)

Presented in 2019 at the 33rd Conference on Neural Information Processing Systems (NeurIPS 2019) in Vancouver, Canada, by Jinsung Yoon and others [23], TimeGANs aim to solve the problem by capturing the potentially complex dynamics of temporal variables. Specifically, in modeling multivariate sequential data: $\mathbf{X_{1:T}=(X_1,\ldots,X_T)}$, we also want to accurately capture the conditional distribution $\mathbf{p(X_t\,|X_{1:t-1})}$ of temporal transitions. The problem can be described as follows: in general, data has a static component, i.e., characteristics that do not vary over time, and a dynamic component, formed by data that changes over time: for example, in the case of personal data, the name, first name, and gender are static, whereas age and health status are dynamic. We call S the vector space containing the static characteristics and $\boldsymbol{X}$ the vector space of dynamic characteristics, i.e., $S \in S$ and $\mathbf{X} \in \mathbf{x}$ are two random vectors whose specific values are denoted by $\boldsymbol{s}$ and $\boldsymbol{x}$. Within a certain time interval T, we can consider the set:

$\mathbf{(S, X_{1:T})}$

To which a common probability p is associated, which also includes the random variable T; we therefore call the set of training data:

$$D = (\boldsymbol{s}_n\,, x_n)_1^{T_n}$$

Our goal is to produce, from the training data, the best possible approximation $\hat{p}(\mathrm{S}, X_{1:T})$ of the original density $p(\mathrm{S}, X_{1:T})$

It is difficult to achieve this with a standard GAN structure. This is why we also use the autoregressive decomposition of the joint equation: $p(\mathrm{S}, X_{1:T}) = p(S)\prod_t p(X_t|\, S, X_{1:t-1})$

To specifically focus on the conditionals, which allows us to achieve the complementary and simpler objective of learning a density: $\hat{p}(X_t|\,\mathrm{S}, X_{1:t-1})$ at any time t. Two distinct objectives arise from this decision. The first can be expressed as: $\min_{\hat{p}} D(\,p(\mathrm{S}, X_{T-1})\|\,\hat{p}(S, X_{T-1}))$ (16)

where D is a measure of the distance between the two distributions, and is global.

The second: $\min_{\hat{p}} D(\,p(X_t|\,\mathrm{S}, X_{1:t-1})\|\,\hat{p}(X_t|\,S, X_{1:t-1}))$ (17) which, instead, measures locally the point-to-point distance between the original sequence at time t and the generated sequence. From there, we determine the objective that combines the GAN, related to expression (16), which corresponds to the Jensen-Shannon divergence, and a supervised training through maximum likelihood (ML) that uses the original data and corresponds to the Kullback-Leibler divergence, related to expression (17).

4.2.1 Architecture of the TimeGAN Model

The TimeGAN consists of four network elements: an embedding function, a recovery function, a sequence generator, and a sequence discriminator. The main idea is that the auto-encoding components (the first two) are trained jointly with the adversarial components (the last two), so that TimeGAN simultaneously learns to encode features, generate representations, and iterate over time. The embedding network provides the latent space, the adversarial network operates in this space, and the latent dynamics of real and synthetic data are synchronized by a supervised loss. We describe each of these elements in turn (Figure 6).

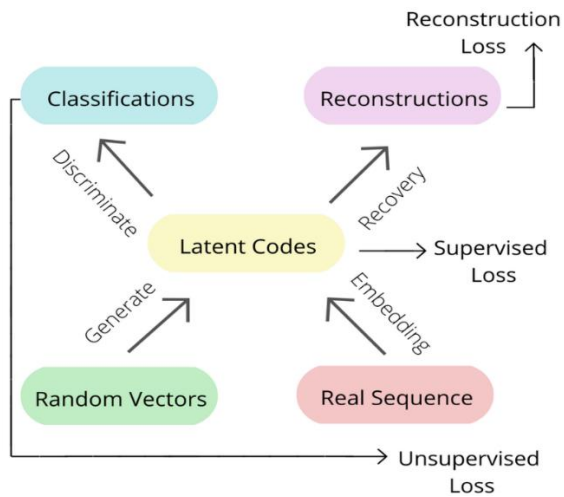

Figure 6: Architecture of TimeGAN

The two components of embedding and recovery are autoencoders designed to conveniently map the latent space, so that the adversarial network can learn the temporal relationships between the data through dimensionality reduction. Recalling that the specific values of the static and dynamic characteristics are respectively $\boldsymbol{s}$ and $\boldsymbol{x}$, and calling $\mathbf{h_s}$ and $\mathbf{h_t}$ the components of the latent spaces (static and temporal), the embedding network will be a (recurrent) network that performs the following operations:

$$\boldsymbol{h_S} = \boldsymbol{e_S}(\boldsymbol{s}) \quad ; \quad \boldsymbol{h_t} = \boldsymbol{e_X}(\boldsymbol{h_S}\,,\ \boldsymbol{h_{t-1}}\ ,\ \boldsymbol{x_t}\ )$$

Where $\mathbf{e_s}$ and $\mathbf{e_x}$ are two networks, the first for static functions, the second for temporal functions. The recovery network will need to reconstruct the static characteristics $\bar{\boldsymbol{s}}$ and temporal characteristics $\bar{\boldsymbol{x}}_{\boldsymbol{1:T}}$ implemented with two networks $\boldsymbol{r_S}$ and $\boldsymbol{r_X}$:

$$\bar{\boldsymbol{s}} = \boldsymbol{r_S}(\boldsymbol{h_S}) \quad ; \quad \bar{\boldsymbol{x}}_{\boldsymbol{t}} = \boldsymbol{r_X}(\boldsymbol{h_t})$$

implemented by feedforward networks.

4.2.2 Sequence Generator and Discriminator

Here too, the generator and discriminator operate on static and dynamic data. For the generator:

$$\widehat{\boldsymbol{h}}_{\boldsymbol{S}} = \boldsymbol{g_S}(\boldsymbol{z_S}) \quad ; \quad \widehat{\boldsymbol{h}}_{\boldsymbol{t}} = \boldsymbol{g_X}(\widehat{\boldsymbol{h}}_{\boldsymbol{S}}\,, \widehat{\boldsymbol{h}}_{\boldsymbol{t-1}}, \boldsymbol{z_t})$$

where $\mathbf{g_s}$ is a generator network for static functions and $\mathbf{g_x}$ is a recurrent network for temporal functions. The random vector $\mathbf{z_s}$ can be sampled from a distribution of choice, and $\mathbf{z_t}$ follows a stochastic process; here we use the Gaussian distribution and the Wiener process.

For the discriminator: $\tilde{y}_S = d_S(\widehat{\boldsymbol{h}}_{\boldsymbol{S}})$; $\tilde{y}_t = d_X(\overleftarrow{u}_t\,, \vec{u}_t)$

Where $\vec{u}_t = \vec{c}_X(\hat{h}_S\,, \hat{h}_t\,,\ \vec{u}_{t-1})$ and $\overleftarrow{u}_t = \overleftarrow{c}_X(\hat{h}_S\,, \hat{h}_t\,,\ \overleftarrow{u}_{t+1})$ represent the forward and backward hidden state sequences, respectively the recurrent functions, $\vec{c}_X\,, \overleftarrow{c}_X$ and the output layer classification functions $d_S\,, d_X$.

4.2.3 Training Scheme

For training, we first have the "reconstruction loss" with which we train the integration and reconstruction networks for the reconstructions $\tilde{\boldsymbol{s}}\,, \bar{\boldsymbol{X}}_{\boldsymbol{1:T}}$ of the original data $\boldsymbol{s}\,,\ \boldsymbol{X}_{\boldsymbol{1:T}}$from the latent representations $\boldsymbol{h_S}\,,\ \boldsymbol{h}_{\boldsymbol{1:T}}$.

$$\mathcal{L}_{\boldsymbol{R}} = \mathbb{E}_{s,X_{1:T\sim p}}\left[\ \|s-\tilde{s}\|_2 + \sum \|X_t - \tilde{X}_t\|_2\ \right] \qquad (18)$$

Secondly, we have the typical unsupervised loss of the GAN, in which the probability of giving correct classifications is maximized for the discriminator, or minimized for the generator.

$$\mathcal{L}_U = \mathbb{E}_{S,X_{1:T\sim p}}\left[\log y_S + \sum_t \log y_t\right] + \mathbb{E}_{S,X_{1:T\sim p}}\left[\log(1-\hat{y}_S) + \sum_t \log(1-\hat{y}_t)\right] \quad (19)$$

To achieve greater adherence to the conditional distributions of the data, an additional loss is introduced to train the generative network in supervised mode, in which the generator receives sequences derived from the integration of real data to generate the next vector. Gradients can now be calculated on a loss that captures the gap between the distributions:

$p(H_t \mid H_S, H_{1:t-1})$ et $\hat{p}(H_t \mid H_S, H_{1:t-1})$ (supervised loss).

$$\mathcal{L}_S = \mathbb{E}_{s,X_{1:T}\sim p}[\sum_t \| h_t - g_X(h_S, h_{t-1}, z_t) \|] \quad (20)$$

During training, the difference between the latent vector provided at the next step by the integration network and the latent vector synthetically generated by the generator is evaluated. $\mathcal{L}_U$ checks that the generator produces plausible sequences for the discriminator, while $\mathcal{L}_S$ checks the progressiveness of the transactions.

4.2.4 Optimization

For the recovery network parameters, θ_g for the generator parameters, and θ_d for the discriminator parameters.

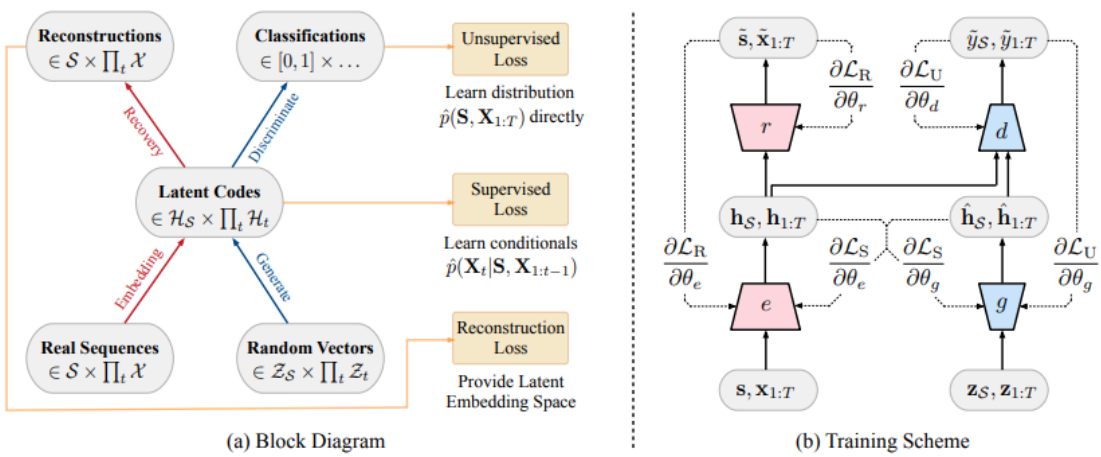

Figure 7: TimeGAN Training Scheme

The solid lines indicate the forward propagation of data, and the dotted lines indicate the backpropagation of gradients. The learning function for the first two networks uses both the reconstruction loss and the supervised loss through a hyperparameter λ>0 that balances the two losses.

$$\min_{\theta_e, \theta_r} (\lambda \mathcal{L}_S + \mathcal{L}_R) \quad (21)$$

The learning function for the generator and discriminator networks uses both the inversion of the loss through a hyperparameter η≥0 to balance the two losses.

$$\min_{\theta_g} (\eta \mathcal{L}_S + \max_{\theta_d} \mathcal{L}_U) \quad (22)$$

In practice, TimeGAN is not particularly sensitive to hyperparameters: for all experiments, the following values were fixed: λ=1 and η=10.

5. Implementation of the Hybrid Prediction Model with TimeGAN

Time series data are sequentially structured, allowing for trend and seasonality analysis. TimeGAN, a Generative Adversarial Network (GAN) model, generates synthetic time series data by learning temporal dependencies from real data. Our hybrid approach integrate TimeGAN to create synthetic datasets for training three time series prediction models: LSTM, BiLSTM, and GRU, with optimized parameters and architectures.

This method enhances predictive performance by combining data augmentation with the strengths of RNN variants (LSTM, BiLSTM, and GRU). The goal is to determine the most efficient model for hourly precipitation forecasting, evaluated using specific performance metrics.

- Mean Squared Error (MSE): $MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \bar{Y}_i)^2$
- Mean Absolute Error (MAE): $MAE = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \hat{Y}_i|$

-Root Mean Squared Error (RMSE): $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2}$

5.1 Preprocessing ERA5 Data

Since ERA5 data are reanalysis data derived from various meteorological sources, they are already preprocessed and do not contain missing values. The preprocessing methods applied here are specific to the requirements of deep learning models. To facilitate network learning, we scaled the data to a range between 0 and 1 using Min-Max normalization.

5.2 TimeGAN Training Data

We use a sliding window of size 24 applied across the dataset, shifting one position at a time. This process generates a series of 2D matrices, each with a length of 24 and containing all column features.
In the ERA5 dataset, there are 4,368 rows. By sampling every 24 rows, we obtain 4,368 entries, each consisting of 24 rows and 11 features. These entries can be randomly shuffled to make them independent and identically distributed.
As a result, we created a dataset with dimensions [4368, (24, 11)], where each of the 4,368 instances contains 24 rows (also referred to as time steps) and 11 features.

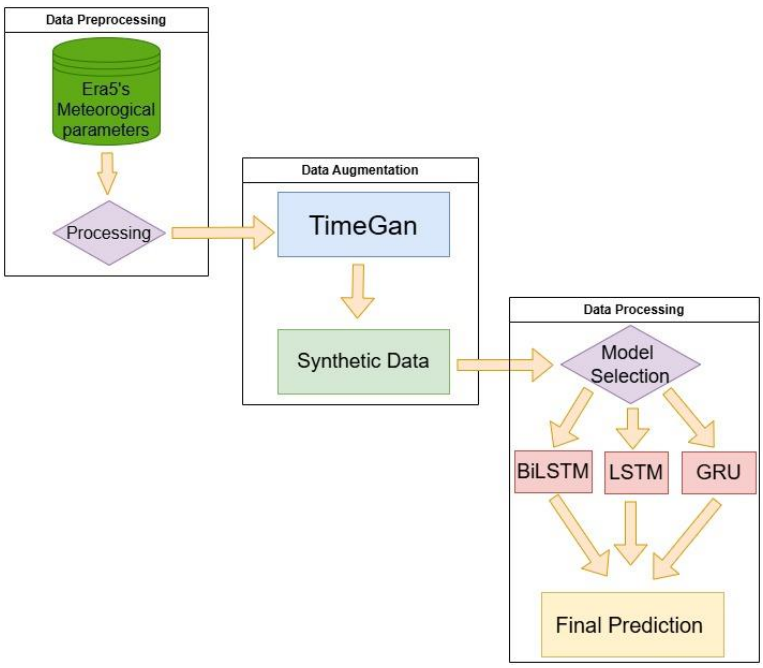

Figure 8: Hybrid Prediction Model with TimeGAN Scheme

5.3 Creation of lag features for prediction

Lag features are commonly used in data science to forecast time series with traditional datasets. When predicting future values of a variable, its past values tend to be predictive. Additionally, the past values of other predictive features can also be useful for our forecasts. Thus, in forecasting, it is common to create lag features from time series and use them as inputs to machine learning or forecasting models.

6. Results

6.1 Presentation of TimeGAN Training Results

6.1.1 2D Visualization: A Qualitative Evaluation of Diversity

To visualize the real and synthetic time series with 24-time steps and 11 features, we reduce their dimensionality so that they can be represented in two dimensions. To this end, we sample 250 normalized sequences, each with 11 features, and reshape them to obtain data with dimensions of 2750 × 24.

Figure 9 presents the PCA and t-SNE results for a qualitative evaluation of the similarity between the distributions of real and synthetic data. Both methods reveal remarkably similar patterns and significant overlap, suggesting that the synthetic data capture important aspects of the characteristics of the real data.

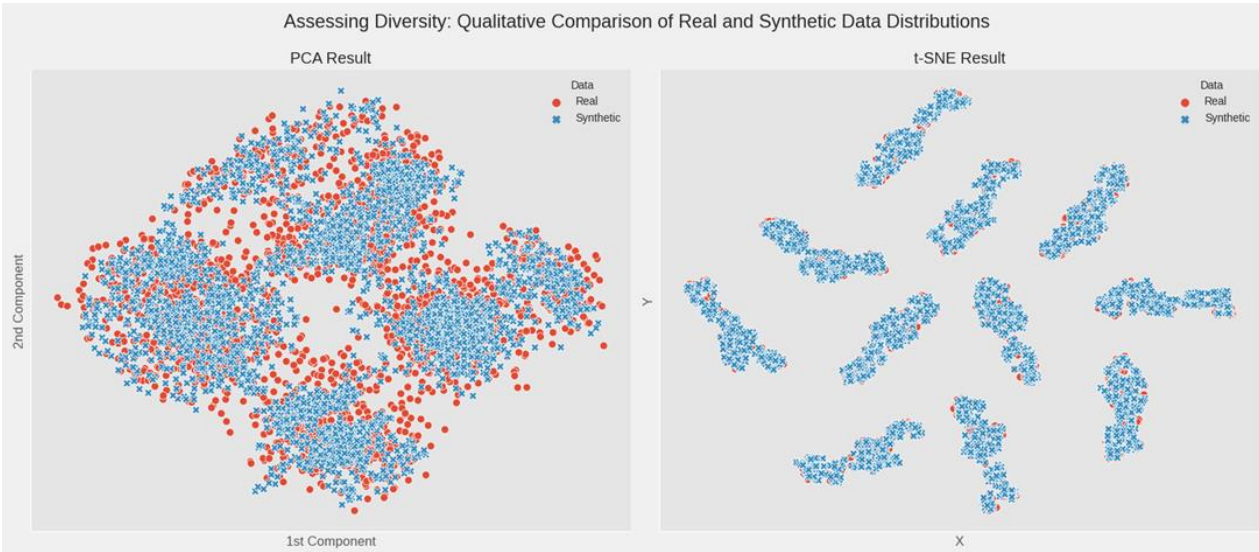

Figure 9: PCA and t-SNE Visualization

6.1.2 Time Series Classification: A Quantitative Evaluation of Fidelity

Visualization only provides a qualitative impression. For a quantitative evaluation of the fidelity of the synthetic data, we train a time series classifier to distinguish real data from fake data and evaluate its performance on a held-out test set. Specifically, we select the first 80% of the sliding sequences for training and the last 20% as the test set.

We then create a simple RNN with six units that receives mini-batches of real and synthetic time series of shape 24 × 11 and uses a sigmoid activation. We optimize it using binary cross-entropy loss and the Adam optimizer while monitoring AUC and precision metrics.

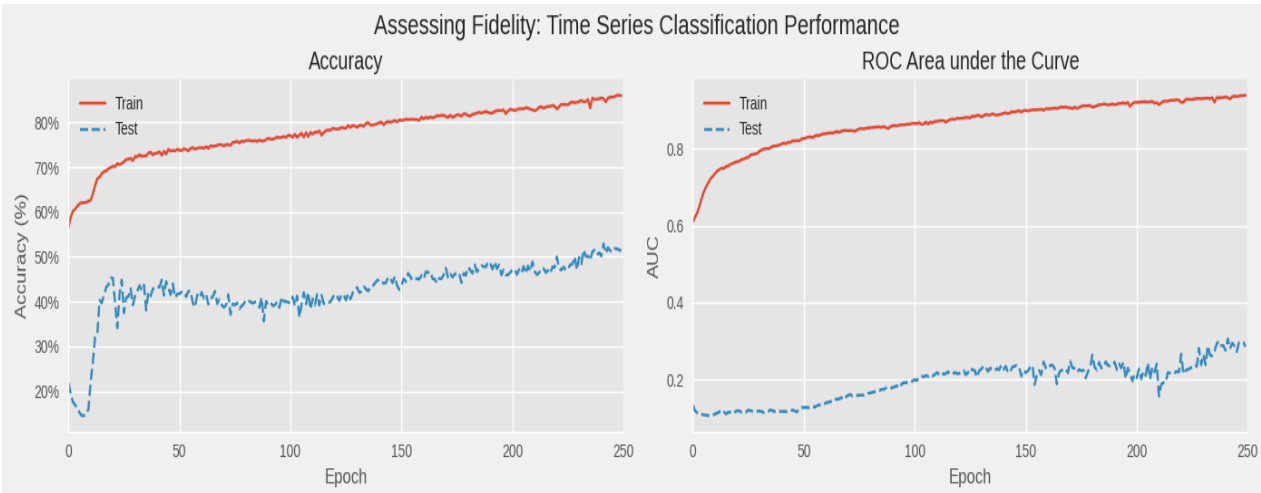

Figure 10: Accuracy Visualization and ROC Curves (AUC)

6.2 Training on Synthetic Data, Testing on Real Data: Evaluating Utility

Finally, we want to determine the extent to which the synthetic data is useful for solving a prediction problem. To do this, we train a time series prediction model alternately on synthetic and real data to predict the next time step and compare the performances on a test set created from the real data.

We create a single-layer RNN with 12 GRU units that predicts the last time step for each of the 11 features, resulting in 11 linear output units. The model uses the Adam optimizer to minimize the mean absolute error (MAE).

We train the model twice once using synthetic data for training and once using real data while evaluating out-of-sample performance on the real test set. Training on synthetic data follows the same procedure as training on real data. Since the training was conducted using the cross-validation sampling method with a rolling/sliding window split and a daily time window, this figure shows the evolution of actual and predicted precipitation over the months across the test dataset. Thus, we can see that the monthly predictions closely follow the actual values, allowing us to assess the prediction quality of the hybrid TimeGAN-LSTM model (Figure 11).

Scatter plot with actual precipitation versus predicted precipitation, along with a red line indicating the ideal scenario where the predicted precipitation perfectly matches the actual precipitation. This visualization helps assess the accuracy of the LSTM model through visual inspection. Additionally, the plot provides a quick understanding of how closely the predictions align with the actual values. It is worth noting that we use the test dataset from 07/10/2021 at 12 PM, providing a realistic evaluation of the model's performance under real-world conditions (Figure 12).

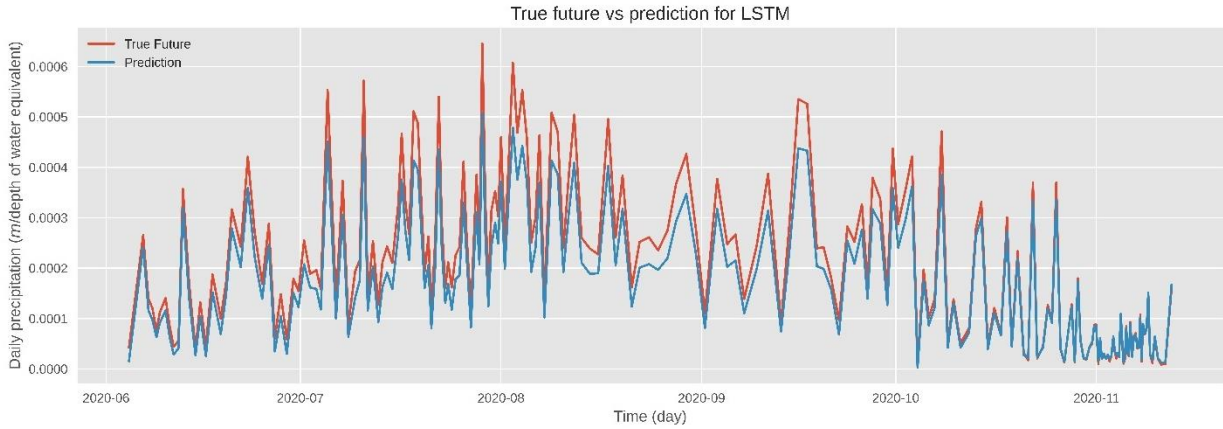

Figure 11: Evolution of Actual and Predicted Precipitation on the Test Dataset with the Hybrid TimeGAN-LSTM Model

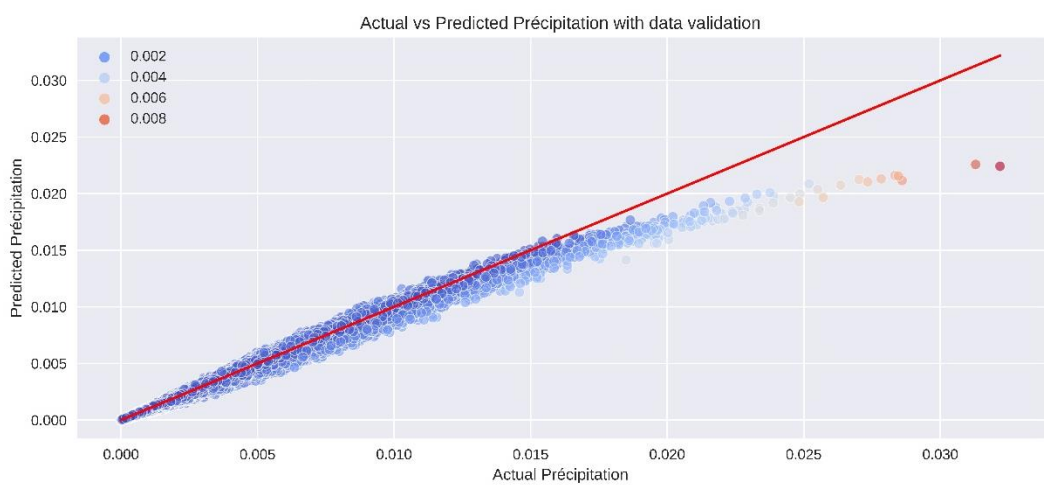

Figure 12: Scatter Plot of Actual vs. Predicted Precipitation with Ideal Reference Line for LSTM Model Evaluation

In figure 13, we present the actual precipitation and the predicted precipitation from the hybrid model with an hourly time scale on the test data from 07/10/2021 between 00:00 and 12:00. By visually inspecting this figure, we can see that the hybrid model is capable of making hourly precipitation predictions.

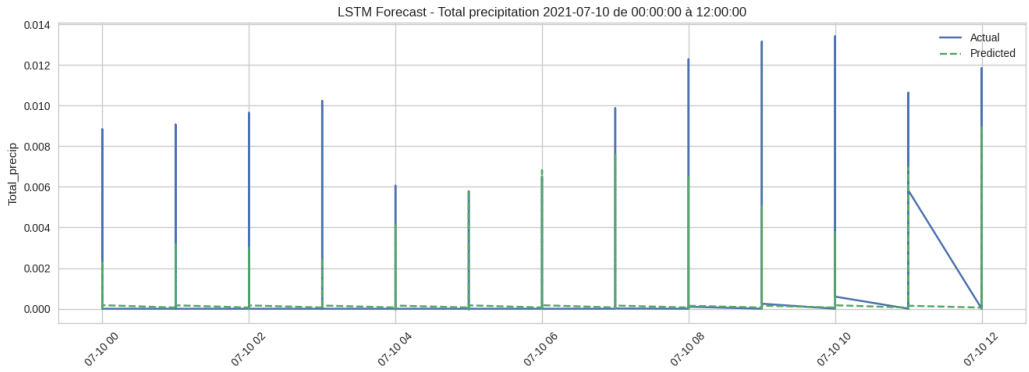

Figure 13: Hourly Actual and Predicted Precipitation from the Hybrid Model on Test Data (07/10/2021, 00:00–12:00)

The model demonstrates exceptional performance, with a Mean Squared Error (MSE) of 2.099×10^{-9}, a Mean Absolute Error (MAE) of 3.500×10^{-5}, and a Root Mean Squared Error (RMSE) of 4.582×10^{-5}, collectively validating the high precision of the TimeGAN-LSTM hybrid approach across both monthly trends and hourly precipitation forecasts.

7. Conclusion

The study demonstrates that synthetic data generated by TimeGAN can effectively replicate the temporal dependencies and characteristics of real time series data, making it a valuable tool for data augmentation. By combining TimeGAN with advanced RNN variants (LSTM, BiLSTM, and GRU), the hybrid approach achieves accurate and reliable short-term hourly rainfall predictions. The qualitative and quantitative evaluations confirm that the synthetic data closely mirrors the distribution and patterns of real data, as evidenced by PCA, t-SNE, and classification metrics. Training predictive models on synthetic data yields performance comparable to or better than training on real data, highlighting the utility of synthetic data in scenarios with limited or insufficient real data. The use of TimeGAN-generated data addresses challenges such as data scarcity and overfitting, particularly in regions like West Africa with highly variable climatic conditions. The hybrid model's ability to predict hourly rainfall has significant implications for water resource management, agriculture, and disaster prevention in vulnerable regions. The study underscores the importance of leveraging generative models to enhance the robustness and generalization capabilities of time series forecasting models. The results suggest that synthetic data can serve as a viable alternative to real data for training machine learning models, especially in domains where data collection is costly or challenging. Future work could explore the application of this hybrid approach to other time series prediction tasks, such as energy demand forecasting or financial market analysis. Overall, this research advances the field of time series forecasting by demonstrating the potential of synthetic data and hybrid models to improve prediction accuracy and support decision-making in critical applications.

References

[1] Henderson, M., Thompson, S., & James, A. (2018). Climate variability and its impact on water resources in West Africa. Environmental Monitoring and Assessment, 190(3), 178.
[2] Harrington, M., & Mahfouf, J. (2020). Short-term precipitation forecasting: A case study from West Africa. Atmospheric Research, 240, 104907.
[3] McCarty, D., Zhao, Y., & Zhang, H. (2019). Advancements in short-term weather prediction for disaster mitigation. Environmental Hazards, 18(2), 210-225.
[4] Diouf, D., & Ndiaye, A. (2021). Challenges of weather prediction in West Africa: Application of machine learning models. Climate Change and Development, 32(2), 109-122.
[5] Pérez, E., López, A., & García, A. (2017). Short-term precipitation prediction: Benefits for agricultural planning. Agricultural Meteorology, 23(4), 211-220.
[6] Patel, R., & Sharma, K. (2020). Impact of weather prediction on agriculture: A case study of West Africa. Agricultural Systems, 178, 102758.

[7] Srinivasan, R., Nair, A., & Sharma, T. (2018). Management of water reservoirs using weather forecasting models. Water Resources Management, 32(7), 2357-2369.
[8] Chouhan, P., & Sharma, R. (2019). Improved rainfall prediction using generative adversarial networks for synthetic data augmentation. Environmental Modelling & Software, 116, 37-45.
[9] Li, F., Li, J., & Zhao, W. (2020). Data augmentation using TimGAN for time series forecasting. Journal of Machine Learning Research, 21(125), 1-16.
[10] Wang, X., Zhang, S., & Zhao, M. (2019). Time series forecasting of precipitation using deep learning models: A comparative study. Water Resources Management, 33(10), 3399-3412.
[11] Shi, Y., Sun, Q., & Wang, J. (2021). Precipitation prediction using a hybrid LSTM-GRU model: A case study of East Asia. Journal of Hydrology, 590, 125679.
[12] Zhao, Y., Liu, D., & Wu, X. (2020). A hybrid LSTM and GAN model for precipitation prediction under changing climate conditions. Environmental Modelling & Software, 132, 104766.
[13] Yoon, J., Kim, S., & Choi, Y. (2020). Improved short-term weather forecasting using a hybrid model of LSTM and TimGAN. Journal of Climate, 33(14), 4876-4893.
[14] Dai, J., Zhang, Z., & Li, B. (2021). A hybrid model combining LSTM and GAN for short-term precipitation forecasting. Atmospheric Research, 246, 105128.
[15] Zhang, Jianfeng, et al. (2018). Developing a long short-term memory (LSTM) based model for predicting water table depth in agricultural areas. J. Hydrol. 561, 918–929.
[16] Zhao, Y., Liu, D., & Wu, X. (2020). A hybrid LSTM and GAN model for precipitation prediction under changing climate conditions. Environmental Modelling & Software, 132, 104766.
[17] Sak, H., Senior, A., & Beaufays, F. (2014). Long short-term memory recurrent neural network architectures for large scale acoustic modeling. Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH, 338-342.
[18] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.
[19] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research, 270(2), 654-669.
[20] Cui Z, Ke R, Pu Z, et al. (2018). Deep bidirectional and unidirectional LSTM recurrent neural network for network-wide traffic speed prediction. arXiv preprint arXiv:1801.02143.
[21] Kim J, Moon N. (2019). BiLSTM model based on multivariate time series data in multiple field for forecasting trading area. Journal of Ambient Intelligence and Humanized Computing, 1-10.
[22] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.
[23] Yoon, J., Jarrett, D., van der Schaar, M. (2019). Time-series Generative Adversarial Networks. Neural Information Processing Systems (NeurIPS).